Network Packet Crafting Using Python and Scapy

Category: Network Security | Network Programming
Tools: Python, Scapy, Kali Linux
Protocols: DNS, UDP, ICMP, TCP
Devarth Patel

1. Overview

This lab focuses on using Python and Scapy to construct, transmit, and inspect network packets within a Kali Linux environment.
Scapy is a Python-based packet manipulation library that allows users to build custom network packets, transmit them, receive responses, and examine protocol information.
For this lab, I developed a Python script named my_scapy.py to perform DNS resolution, construct an ICMP echo-request packet, and transmit a TCP SYN packet.
This provided practical experience with network programming and helped me understand how different protocols operate at the packet level.

2. Objectives

The primary objectives of this lab were to:

- Develop a Python script using the Scapy library.
- Construct and transmit a DNS query to resolve a hostname.
- Extract an IP address from a DNS response.
- Construct an ICMP echo-request packet and inspect its structure.
- Construct and transmit a TCP SYN packet targeting port 80.
- Examine network packet summaries and responses.
- Understand how Python-based packet crafting can support network analysis and security testing.

3. Tools and Environment

Component	**Description**
Operating System	Kali Linux
Programming Language	Python 3
Packet Manipulation Library	Scapy
Text Editor	GNU nano
DNS Resolver	8.8.8.8
Target Hostname	scanme.nmap.org
Target TCP Port	80
Protocols	DNS, UDP, ICMP, TCP

In my coursework, we used scanme.nmap.org, a public host provided by Nmap for authorized testing demonstrations. The results documented here are observations from the host.

4. Methodology

4.1 Developing the Python Script

I created a Python script called my_scapy.py using GNU nano in Kali Linux. The script imported Scapy's networking functionality:

```
from scapy.all import *
import re
```

The Scapy library gives access to packet layers and functions like IP(), UDP(), DNS(), ICMP(), TCP(), and sr1(). The script was then organized into three primary operations:

1. Constructing and sending a DNS query.
2. Constructing and inspecting an ICMP echo-request packet.
3. Constructing and sending a TCP SYN packet.

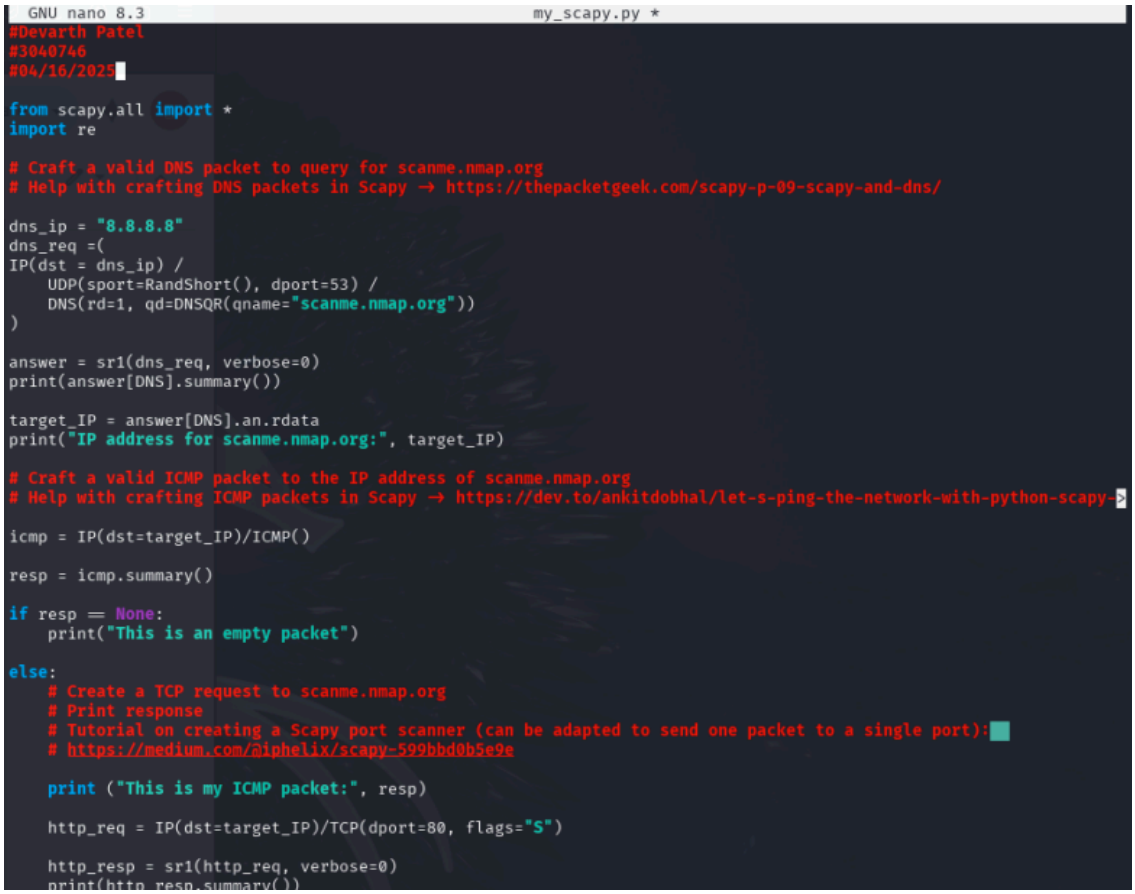


Figure 1: Original Python script written in Kali Linux, demonstrating DNS, ICMP, and TCP packet construction using Scapy.

4.2 DNS Query Construction and Resolution

The first portion of the script was designed to resolve the hostname scanme.nmap.org into an IP address. I configured Google's public DNS resolver as the destination and constructed a DNS query using Scapy:

```
dns_ip = "8.8.8.8"
dns_req = (
   IP(dest=dns_ip) /
   UDP(sport=RandShort(), dport=53) /
   DNS(rd=1, qd=DNSQR(qname="scanme.nmap.org"))
)
```

The packet contained three protocol layers:

- IP: Specifies the destination IP address of the DNS resolver.
- UDP: Uses a randomly selected source port and destination port 53.
- DNS: Creates a recursive DNS query for the specified hostname.

The / operator in Scapy combines protocol layers into a single packet.
After constructing the request, I used sr1() to transmit the packet and receive the first response:

```
answer = sr1(dns_req, verbose=0)
print(answer[DNS].summary())

target_IP = answer[DNS].an.rdata
print("IP address for scanme.nmap.org:", target_IP)
```

The script accessed the DNS answer section and extracted the IP address associated with the requested hostname. The recorded terminal output showed:

```
45.33.32.156
```

This demonstrated how DNS queries can be generated and processed programmatically using Scapy.

4.3 ICMP Echo-Request Packet Construction

After resolving the hostname, I used the returned IP address to construct an ICMP packet.
ICMP is used for network diagnostics, error reporting, and reachability testing.
The following code created the ICMP echo-request packet:

```
icmp = IP(dest=target_IP)/ICMP()
resp = icmp.summary()
```

The packet consisted of an IP layer containing an ICMP message. The ICMP() constructor creates an echo-request packet by default Then, I used the summary() method to obtain a readable description of the constructed packet.
The script then checked the summary value:

```
if resp == None:
    print("This is an empty packet")
else:
    print("This is my ICMP packet:", resp)
```

The terminal displayed a packet summary identifying the ICMP echo request. This portion of the script constructed and summarized an ICMP packet but did not transmit it. Hence, the output demonstrates successful packet construction rather than confirmed network reachability or an ICMP echo reply.

4.4 TCP SYN Packet Construction and Transmission

The final portion of the script focused on creating a TCP SYN packet.
TCP is a transport protocol that normally establishes a connection using a three-way handshake. The first part of the handshake involves transmitting a packet with the SYN flag enabled.
I used Scapy to make a TCP SYN packet directed toward port 80:

```
http_req = IP(dest=target_IP)/TCP(dport=80, flags="S")
```

The packet configuration included:

Parameter	**Value**	**Purpose**
Destination IP	Resolved target IP	Identifies the destination host
Destination Port	80	Targets the conventional HTTP service port
TCP Flags	S	Sets the SYN flag

Even though the variable is named http_req, the packet was a TCP SYN packet rather than a complete HTTP application-layer request. Then I transmitted the packet and attempted to retrieve a response:

```
http_resp = sr1(http_req, verbose=0)
print(http_resp.summary())
```

The sr1() function sends a packet and waits for the first response. The original terminal screenshot shows a TCP SYN packet summary associated with the target IP address and port 80.But, the screenshot doesn't prove a successful TCP three-way handshake.

4.5 Executing the Python Script

After writing the script, I executed it through the Kali Linux terminal using:

```
sudo python3 my_scapy.py
```

The script generated output related to DNS resolution, ICMP packet construction, and TCP packet transmission. The recorded execution showed the resolved IP address of the target hostname and summaries of the constructed packets.



Figure 2: Terminal output from the original Scapy script showing DNS resolution, ICMP echo-request information, and TCP SYN packet details.

5. Results and Findings

Operation	**Protocol**	**Result**
DNS Query	DNS / UDP	Resolved scanme.nmap.org to 45.33.32.156
ICMP Packet Construction	ICMP	Constructed and displayed an echo-request packet
TCP Packet Construction	TCP	Created a SYN packet targeting port 80
TCP Packet Transmission	TCP	Sent the SYN packet using sr1()
Packet Inspection	Scapy	Displayed DNS and packet summary information

The results demonstrated that Python and Scapy can be used to construct and inspect packets across multiple network protocols. It also showed the difference between constructing a packet, transmitting it, and confirming successful communication with a remote system.

6. Cybersecurity Relevance

Packet crafting is a useful skill in cybersecurity because it provides direct control over network packet structures and communication behavior. It also demonstrated these concepts:

- **Network Protocol Analysis:** Constructing IP, UDP, DNS, ICMP, and TCP packets helped reinforce my understanding of protocol layering and individual packet fields.
- **Network Troubleshooting:** Custom packets can be used to investigate network behavior and diagnose communication issues in authorized environments.

- **Security Testing:** Packet manipulation tools can help security professionals test how systems and security controls respond to different network traffic patterns.
- **Python Automation:** Scapy demonstrated how Python can automate network operations that would otherwise require manual interaction with networking tools.
- **Intrusion Detection:** Understanding packet fields and transport-layer behavior provides a foundation for interpreting traffic monitored by intrusion detection systems.

These skills are relevant to network security, security engineering, and security operations.

7. Source Code and Evidence

Source Code

File: my_scapy.py
The original Python script used in this assignment is included in the GitHub repository to provide direct evidence of the packet construction methods used during the exercise.